

Introduction to Programming

EMATM0048- Teaching Session 1A



Session's contents

- Why programming?
- Aspects of languages
- Difference between programming languages
- Python as a programming language

What does a computer do?

- Fundamentally:
 - performs **calculations** : billion calculations per second!
 - **remembers** results 100s of gigabytes of storage!
- What kinds of calculations?
 - **built-in** to the language
 - ones that **you define** as the programmer
- Computers only know what you tell them.

Types of knowledge

- **Declarative knowledge** is statements of fact.
 - someone will win an alpaca ride today
- **Imperative knowledge** is a recipe or “how-to”.
 - Students sign up for raffle
 - William opens an excel sheet
 - William chooses a random number between 1st and nth responder
 - William finds the number in the responders sheet. Winner!

A numerical example

- Square root of a number x is y such that $y*y = x$  Fact!
- Recipe for deducing square root of a number x (say 25)
 - 1) Start with a guess, g
 - 2) If $g*g$ is close enough to x , stop and say g is the answer
 - 3) Otherwise make a new guess by averaging g and x/g
 - 4) Using the new guess, repeat process until close enough

A numerical example

g	$g * g$	x/g	$(g+x/g)/2$
3	9	8.3	5.67

A numerical example

g	$g * g$	x/g	$(g+x/g)/2$
3	9	8.3	5.67
5.67	32.15	4.41	5.04

A numerical example

g	$g * g$	x/g	$(g+x/g)/2$
3	9	8.3	5.67
5.67	32.15	4.41	5.04
5.04	25.04		

What is a recipe?

- Sequence of simple steps
- Flow of control process that specifies when each step is executed
- A means of determining when to stop



A recipe is an algorithm

Image by [Moirra Nazzari](#) from [Pixabay](#)

bristol.ac.uk

Computers are machines

- Stored program computer
 - Machine stores and executes instructions
 - The heart of the computer then becomes a program (called an interpreter)
 - An interpreter can execute any legal set of instructions
 - Both the program and the data it manipulates reside in memory.
 - Who was the first computer programmer?



Photo by [bert sz](#) on [Unsplash](#)

Stored program computer

Sequence of instructions stored inside computer

- built from predefined set of primitive instructions
 - arithmetic and logic
 - simple tests
 - moving data
- special program (interpreter) executes each instruction in order
 - use tests to change flow of control through sequence
 - stop when done

Session's contents

- Why programming?
- Aspects of languages
- Difference between programming languages
- Python as a programming language

Aspects of **ANY** language

Primitive constructs

Syntax

Static Semantics

Semantics

Aspects of languages

Primitive constructs

Natural Language	Programming Language
words	numbers, strings, simple operators

Aspects of languages

Syntax

Natural Language	Programming Language
• “Cat dog boy” not valid • “Cat hugs boy” valid syntactically	• “Hi”5 not syntactically valid • 3.2*5 syntactically valid

Aspects of languages

static semantics
defines which sentences are meaningful

Natural Language	Programming Language
"I are hungry" syntactically valid but static semantic error	3+"hi" static semantic error

Aspects of languages

Semantics
defines the meaning of those
sentences

Natural Language	Programming Language
<i>"Alice saw Bob on the hill with a telescope"</i> . Who has the telescope?	have only one meaning but may not be what programmer intended

Where things can go wrong

- Syntactic errors
 - common and easily caught
- Static semantic errors
 - some languages check for these before running program
 - can cause unpredictable behaviour
- No semantic errors but different meaning than what programmer intended
 - program crashes, stops running
 - program runs forever
 - program gives an answer but different than expected

Programming is an art

- Given a fixed set of ingredients a good chef can make an unbounded number of tasty dishes by combining them in different ways.
- Similarly, given a small fixed set of primitive elements a good programmer can produce an unbounded number of useful programs.
- To create recipes, or sequences of instructions, we need a **programming language** in which to describe these things, a way to give the computer its marching orders.

Session's contents

- Why programming?
- Aspects of languages
- Difference between programming languages
- Python as a programming language

Different programming languages

Low-level vs. high-level

- move 64 bits of data from this location to that location OR
- pop up a menu on the screen (abstract operation)

Different programming languages

General vs. targeted to an application domain

- Whether the primitive operations of the programming language are widely applicable or are fine-tuned to a domain.
- Example: Web development and general purpose programming

Different programming languages

Interpreted vs. compiled

- Whether the sequence of instructions written by the programmer, called **source code**, is executed directly (by an interpreter)
 - Examples of common interpreted languages are PHP, Ruby, Python, and JavaScript.
- Or whether it is first converted (by a compiler) into a sequence of machine-level primitive operations.
 - Examples of pure compiled languages are C, C++, Erlang, Haskell, Rust, and Go.

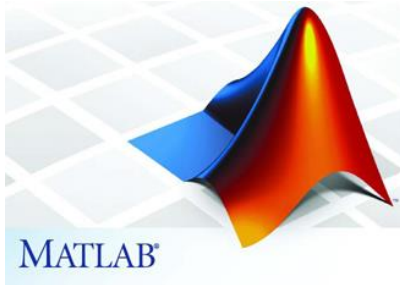
Interpreter vs. Compiler

- An interpreter produces a result from a program, while a compiler produces a program written in assembly language.
- Python is an *interpreted language*
 - Means no compiler, no executable file. The *Python Interpreter* parses the .py file and interprets the code as it goes.
 - There is no prior compilation of source code into machine code.
- What about Java?

Programming for data science

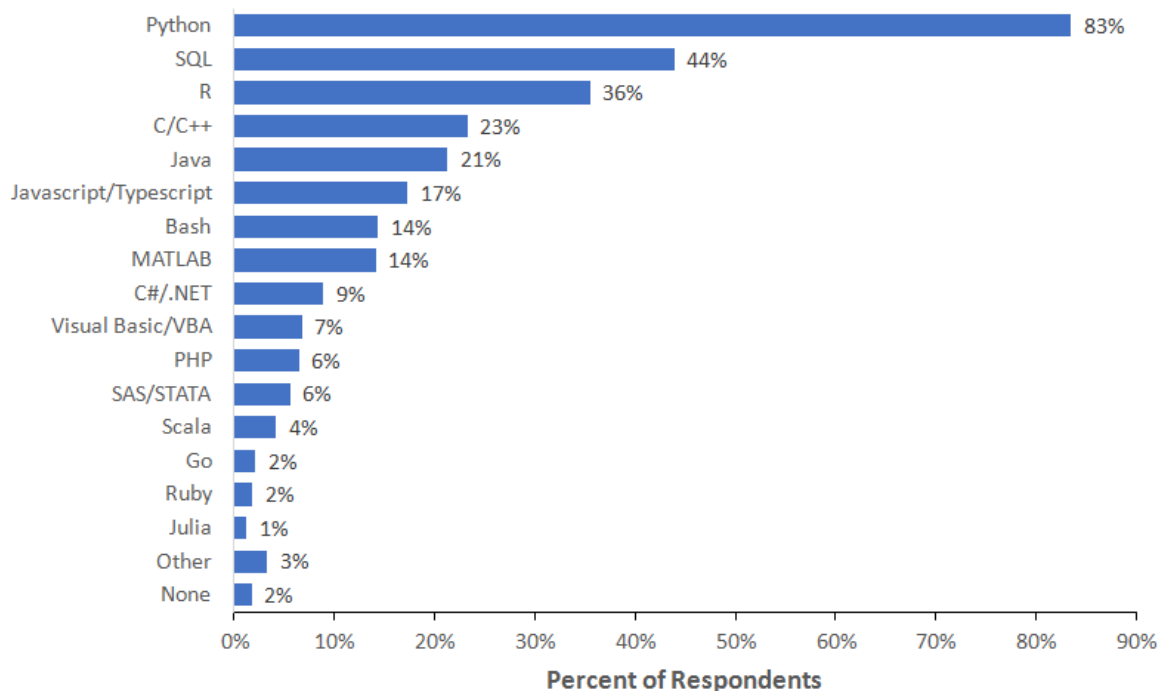


Java™



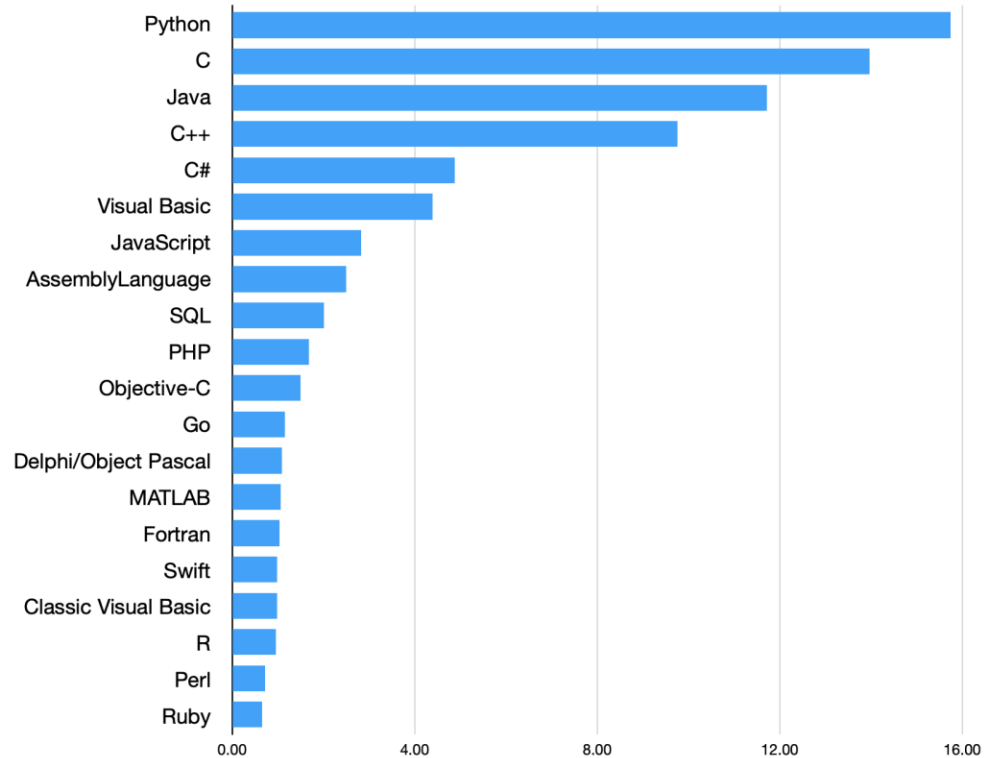
bristol.ac.uk

What programming language do you use on a regular basis?



Note: Data are from the 2018 Kaggle Machine Learning and Data Science Survey. You can learn more about the study here: <http://www.kaggle.com/kaggle/kaggle-survey-2018>. A total of 18827 respondents answered the question.

What are the most popular languages?



Source: <https://www.tiobe.com/tiobe-index/>

Retrieved: September 2022

Also see: Data is Beautiful Website

<https://www.youtube.com/watch?v=Og847HVwRSI>

Session's contents

- Why programming?
- Aspects of languages
- Difference between programming languages
- Python as a programming language

What is Python?

- Python 1 (1994):
 - A middle ground between C and shell scripts.
- Python 2 (2000): Real python
- Python 3 (2008): Improved, more stable version
 - We will use Python 3.8 in this course



Guido Van Rossum
Former Benevolent
Dictator for Life (BDFL)

Python in action!



Python vs. other languages

Java

```
public class HelloWorld{  
    public static void main (string[] args){  
        System.out.println( "Hello world" );  
    }  
}
```

```
$ javac HelloWorld.java  
$ java HelloWorld  
Hello world!
```

Python vs. other languages

C++

```
# include <iostream>
using namespace std;
int main() {
    cout << "Hello world!" << endl;
    return 0;
}
```

```
$ g++ helloWorld.cpp
$ ./a.out
Hello world!
```

Python Vs other languages

Python

```
print("Hello world!")
```

```
$python helloworld.py  
Hello world!
```

Python programs

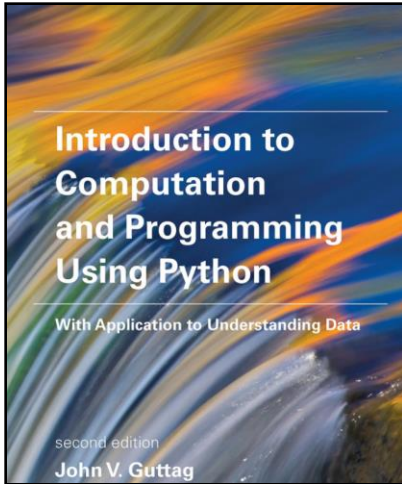
- A program is a sequence of definitions and commands
 - definitions evaluated
 - commands executed by Python interpreter in a shell
- Commands(statements) instruct interpreter to do something
- Can be typed directly in a shell or stored in a file that is read into the shell and evaluated

Takeaways

- Computers need to help, but you have to tell them how, through programming
- Many programming languages are available, with many differences among them
- Python is an interpreter language which we will use for this course.

Readings

Chapter 1: Getting Started

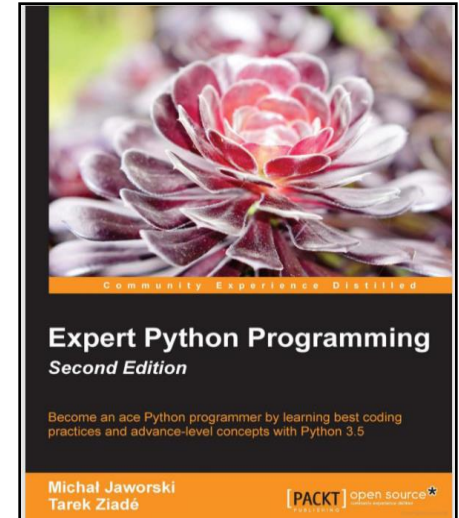


Chapter 1: The way of the program



OPTIONAL

Chapter 1: Computers and Programs



Next session

- The basic elements of Python
- Python modes
- Objects in python
- Variables and assignments